

# 將智慧住宅裝置連結至 Google 助理 | Cloud-to-cloud | Google Home Developers

來源：<https://developers.home.google.com/codelabs/smarthome-washer?hl=zh-tw>

步驟數：8

建構及部署雲端對雲端整合，將虛擬智慧洗衣機整合至 Google 助理。

---

# 目錄

1. [1. 事前準備](#)
2. [2. 開始使用](#)
3. [3. 執行範例應用程式](#)
4. [4. 建立洗衣機](#)
5. [5. 處理指令和查詢](#)
6. [6. 測試整合項目](#)
7. [7. 向 Google 檢舉更新內容](#)
8. [8. 恭喜](#)

步驟 1 · 約 0 分鐘

將智慧住宅裝置連結到 Google 助理

## 程式碼研究室簡介

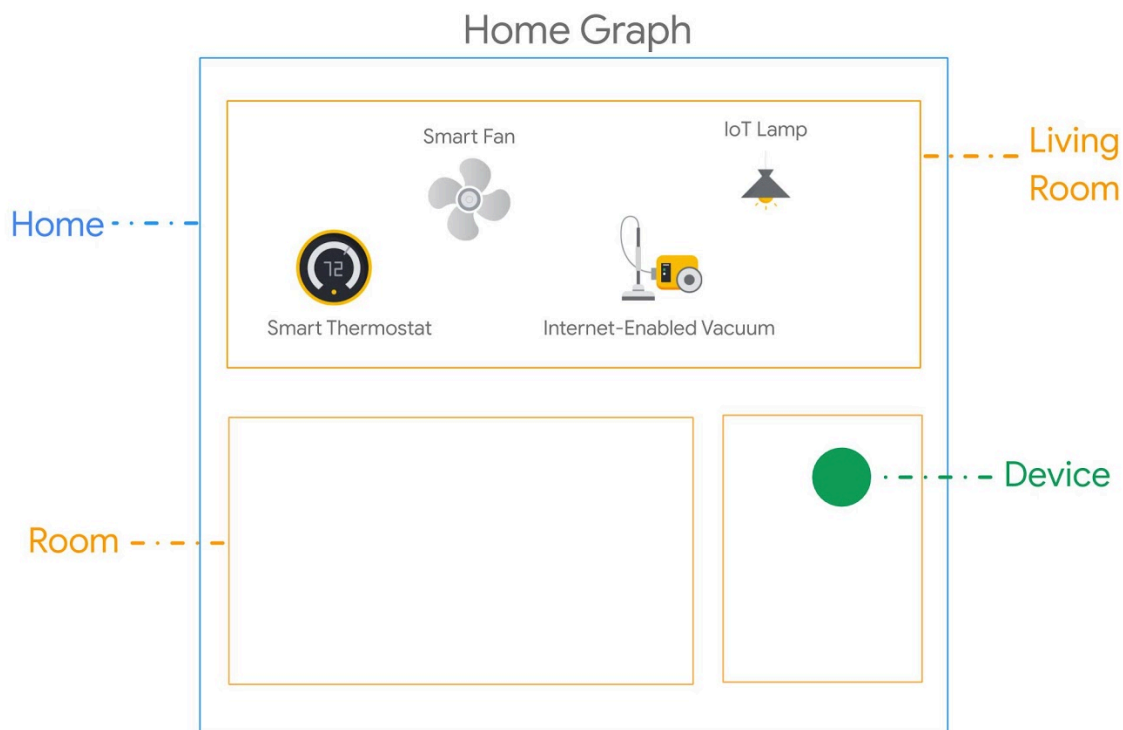
*subject*上次更新時間：5月 12, 2026

*account\_circle*作者：Google 員工

### 1. 事前準備

物聯網 (IoT) 開發人員可以建構[雲端對雲端整合](#)，讓使用者透過 Google Home 應用程式中的觸控手勢和 Google 助理的語音指令控制裝置。

雲端對雲端整合功能會運用 [Home Graph](#) 提供住家和裝置的背景資料，建立住家的邏輯地圖。這樣一來，Google 助理就能更自然地理解使用者在家中不同位置提出的要求。舉例來說，Home Graph 可以儲存客廳的概念，其中包含不同製造商的多種裝置，例如溫度控制器、燈具、風扇和吸塵器。



## 必要條件

- [建立雲端對雲端整合開發人員指南](#)

## 建構項目

在本程式碼研究室中，您將發布可管理虛擬智慧洗衣機的雲端服務，然後建構雲端對雲端整合，並將其連線至 Google 助理。

## 課程內容

- 如何部署智慧型住宅雲端服務
- 如何將服務連結至 Google 助理
- 如何將裝置狀態變更發布至 Google

## 軟硬體需求

- 網路瀏覽器，例如 [Google Chrome](#)
- 已安裝 [Google Home 應用程式](#) 的 iOS 或 Android 裝置
- [Node.js](#) 24 以上版本
- Google Cloud [帳單帳戶](#)
- [Firebase](#) 的 Blaze (即付即用) 方案。

注意：GitHub 存放區中有一些依附元件未列於此處。您必須遵循 GitHub 存放區 `package.json` 檔案中指定的依附元件版本。

步驟 2 · 約 5 分鐘

## 2. 開始使用

### 啟用活動控制項

如要使用 Google 助理，您必須與 Google 分享特定活動記錄。Google 助理需要這項資料才能正常運作，但分享資料的要求並非 SDK 專屬。如要分享這項資料，請建立 [Google 帳戶](#) (如果還沒有帳戶)。您可以使用任何 Google 帳戶，不一定要是開發人員帳戶。

開啟要與 Google 助理搭配使用的 Google 帳戶的[活動控制項頁面](#)。

確認已啟用下列切換開關：

- **網路和應用程式活動** - 此外，請務必勾選「包括 Chrome 歷史記錄，以及採用 Google 服務的網站、應用程式和裝置上的活動記錄」核取方塊。
- **裝置資訊**
- **語音和音訊活動**

您必須在手機和開發人員控制台使用相同的 Google 帳戶，才能完整執行這個專案。

### 建立雲端對雲端整合專案

1. 前往[開發人員控制台](#)。
2. 按一下「建立專案」，輸入專案名稱，然後按一下「建立專案」。

# Create a project

Create a project to get started.

Project name

30 characters max

Create project

Already have a Works with Assistant, cloud, or Local Home SDK project? [Import existing project](#).

## 選取雲端對雲端整合

在開發人員控制台的「專案首頁」中，選取「Cloud-to-cloud」下方的「新增雲端對雲端整合」。

### Cloud-to-cloud

Cloud-connected devices that support smart home Actions. Devices can be controlled through the Google Home app and Google Assistant. [Learn more](#)

[+ Add cloud-to-cloud integration](#)

No cloud-to-cloud integrations yet.

這個專案使用 [Cloud Functions for Firebase](#)，因此您必須將帳單帳戶與專案建立關聯。Actions 專案預設不會建立帳單帳戶。詳情請參閱「[建立新的帳單帳戶](#)」一文。

## 安裝 Firebase CLI

Firebase 指令列介面 (CLI) 可讓您在本地提供網頁應用程式，並將網頁應用程式部署至 Firebase 託管。

如要安裝 CLI，您必須先安裝 [npm](#)，這通常會隨附於 [Node.js](#)。

如要安裝 CLI，請從終端機執行下列 npm 指令：

```
npm install -g firebase-tools
```

如果無法正常運作，建議您[變更 npm 權限](#)。

如要確認 CLI 是否已正確安裝，請執行下列指令：

```
firebase --version
```

執行下列指令，透過 Google 帳戶授權 Firebase CLI：

```
firebase login
```

---

### 3. 執行範例應用程式

現在您已設定開發環境，可以部署入門專案，確認一切設定正確無誤。

#### 取得原始碼

點選下方連結，即可在開發機器上為本程式碼研究室下載範例：

您也可以從指令列複製 GitHub 存放區：

```
git clone https://github.com/google-home/smarthome-washer.git
```

注意：程式碼研究室存放區包含兩個目錄：

- `washer-start`：您要建構的起始程式碼。

- `washer-done`：完成的洗衣機範例應用程式完整程式碼。

如要執行完成的應用程式，您仍需在 [Developer Console](#) 中建立專案。

#### 關於專案

入門專案包含下列子目錄：

- `public`：前端 UI，可輕鬆控制及監控智慧洗衣機的狀態。
- `functions`：透過 Cloud Functions for Firebase 和 Firebase 即時資料庫，管理智慧型洗衣機的雲端服務已全面實作。

提示：尋找 `TODO` 標記。

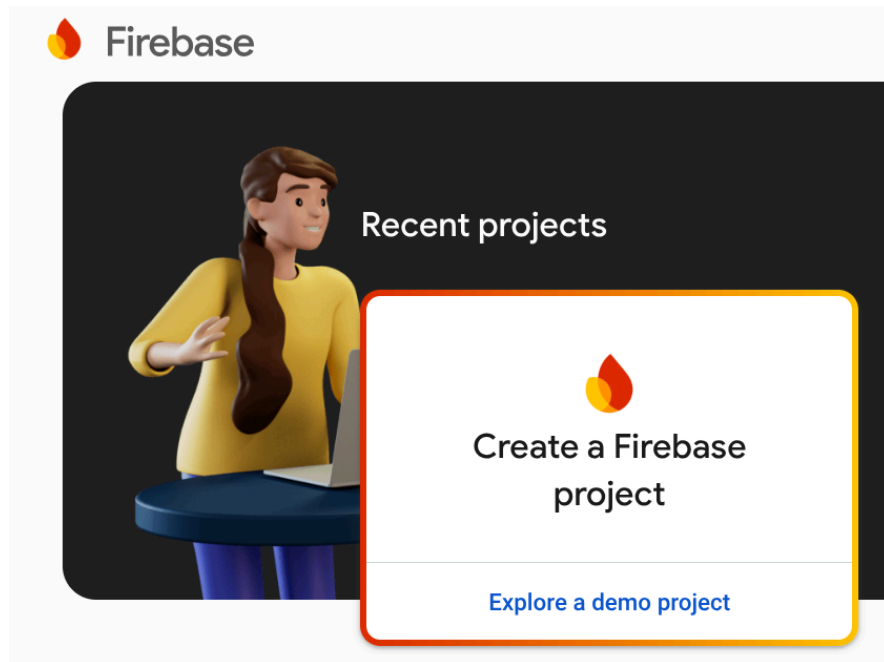
範例專案會提供提示，指出需要變更或新增程式碼的位置。例如：

```
// TODO: Implement full SYNC response
```

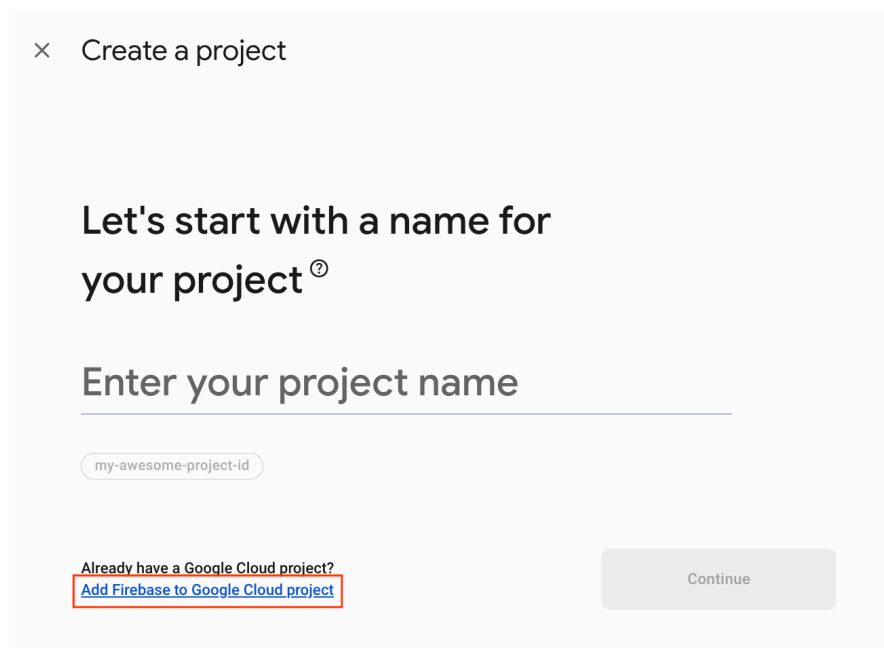
### 將 Firebase 新增至 Google Home 開發人員控制台專案

#### 方法 1：透過 Firebase 控制台

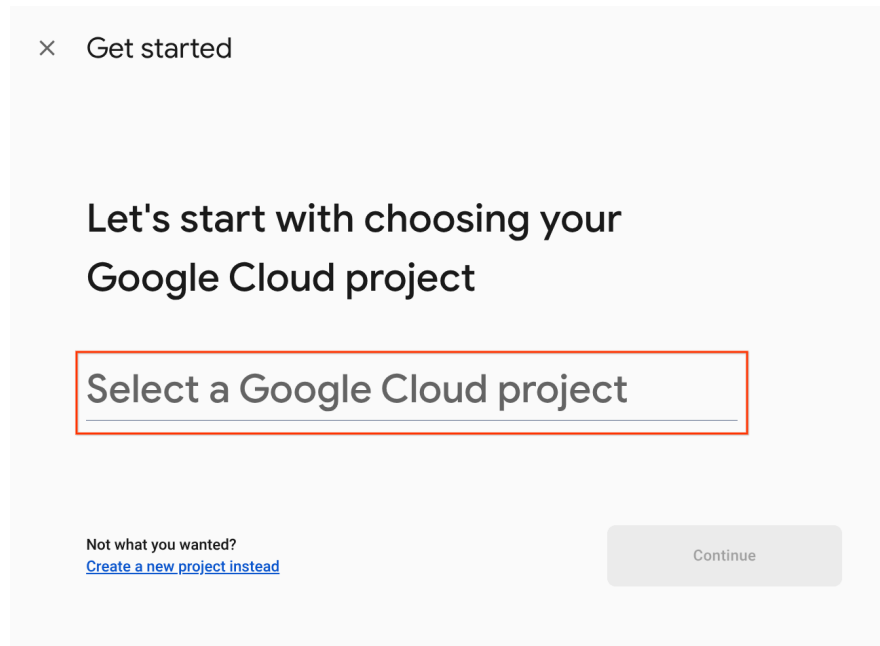
1. 前往 [Firebase](#)。
2. 點選「建立 Firebase 專案」。



3. 在「建立專案」畫面中，按一下「將 Firebase 新增到 Google Cloud 專案」。



4. 在「開始使用」畫面中，選取您剛在 Google Home 開發人員控制台中建立的 Google Cloud 專案，然後按一下「繼續」。

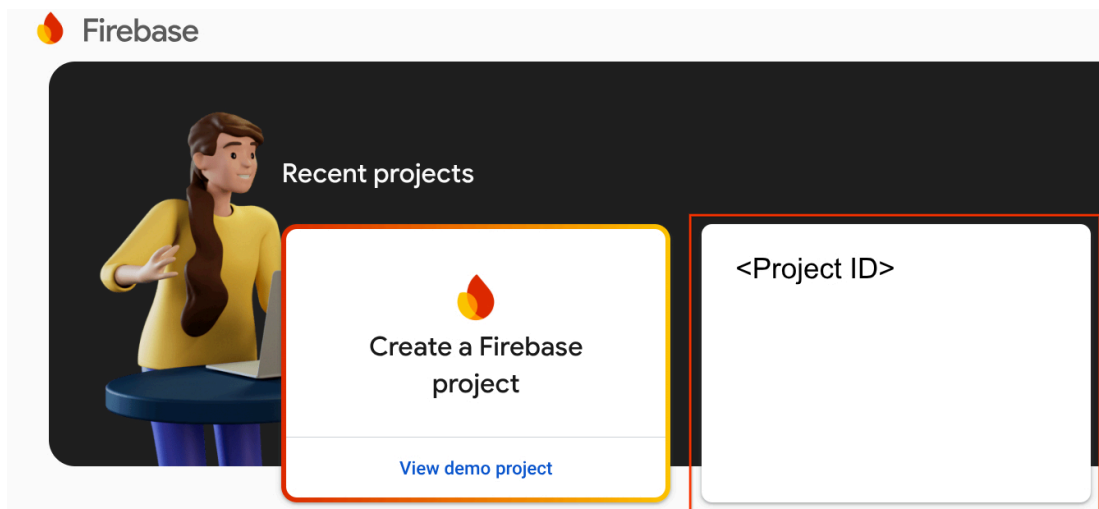


## 方法 2：透過 Firebase CLI

```
firebase projects:addfirebase
```

選取您剛建立的 Google Home 開發人員控制台專案，然後新增 Firebase。

將 Firebase 新增至 Google Home 開發人員控制台專案後，Firebase 控制台就會顯示該專案。Firebase 專案的專案 ID 會與 Google Home 開發人員控制台專案 ID 一致。



## 連結至 Firebase

前往 `washer-start` 目錄，然後使用整合專案設定 Firebase CLI：

```
cd washer-start
firebase use <project-id>
```

提示：如果您不知道 Cloud-to-Cloud 整合的專案 ID，可以在開發人員控制台的「專案首頁」下方找到「專案詳細資料」。

## 設定 Firebase 專案

初始化 Firebase 專案。

```
firebase init
```

選取 CLI 功能、「即時資料庫」和「函式」功能。

```
? Which Firebase features do you want to set up for this directory? Press Space to
select features, then Enter
to confirm your choices. (Press <space> to select, <a> to toggle all, <i> to invert
selection, and <enter> to
proceed)
>( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
( ) Genkit: Setup a new Genkit project with Firebase
(*) Functions: Configure a Cloud Functions directory and its files
( ) App Hosting: Configure an apphosting.yaml file for App Hosting
( ) Hosting: Configure files for Firebase Hosting and (optionally) set up GitHub
Action deploys
( ) Storage: Configure a security rules file for Cloud Storage
( ) Emulators: Set up local emulators for Firebase products
( ) Remote Config: Configure a template file for Remote Config
( ) Extensions: Set up an empty Extensions manifest
(*) Realtime Database: Configure a security rules file for Realtime Database and
(optionally) provision
default instance
( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
```

這會為專案初始化必要的 API 和功能。

系統提示時，請初始化即時資料庫。您可以沿用資料庫執行個體的預設位置。

```
? It seems like you haven't initialized Realtime Database in your project yet. Do you
want to set it up?
Yes

? Please choose the location for your default Realtime Database instance:
us-central1
```

由於您使用的是入門專案程式碼，請選擇安全性規則的預設檔案，並確保不會覆寫現有的資料庫規則檔案。

```
? File database.rules.json already exists. Do you want to overwrite it with the
Realtime Database Security Rules for <project-ID>-default-rtdb from the Firebase
Console?
```

```
No
```

如果重新初始化專案，系統詢問是否要初始化或覆寫程式碼集時，請選取「覆寫」。

```
? Would you like to initialize a new codebase, or overwrite an existing one?
Overwrite
```

設定函式時，請使用預設檔案，並確保不會覆寫專案範例中現有的 **index.js** 和 **package.json** 檔案。

```
? What language would you like to use to write Cloud Functions?
JavaScript
```

```
? Do you want to use ESLint to catch probable bugs and enforce style?
No
```

```
? File functions/package.json already exists. Overwrite?
No
```

```
? File functions/index.js already exists. Overwrite?
No
```

如果重新初始化專案，系統詢問是否要初始化或覆寫 `functions/.gitignore` 時，請選取「No」。

```
? File functions/.gitignore already exists. Overwrite?
No
```

```
? Do you want to install dependencies with npm now?
Yes
```

如果不小心啟用了 ESLint，可以透過以下兩種方法停用：

1. 使用 GUI 前往專案底下的 `../functions` 資料夾，選取隱藏檔案 `.eslintrc.js` 並刪除。請勿與名稱相似的 `.eslintrc.json` 混淆。
2. 使用指令列：

```
cd functions
rm .eslintrc.js
```

## 部署至 Firebase

安裝依附元件並設定專案後，即可首次執行應用程式。

```
firebase deploy
```

您應該會看到下列控制台輸出內容：

...

✓ Deploy complete!

Project Console: <https://console.firebase.google.com/project/<project-id>/overview>

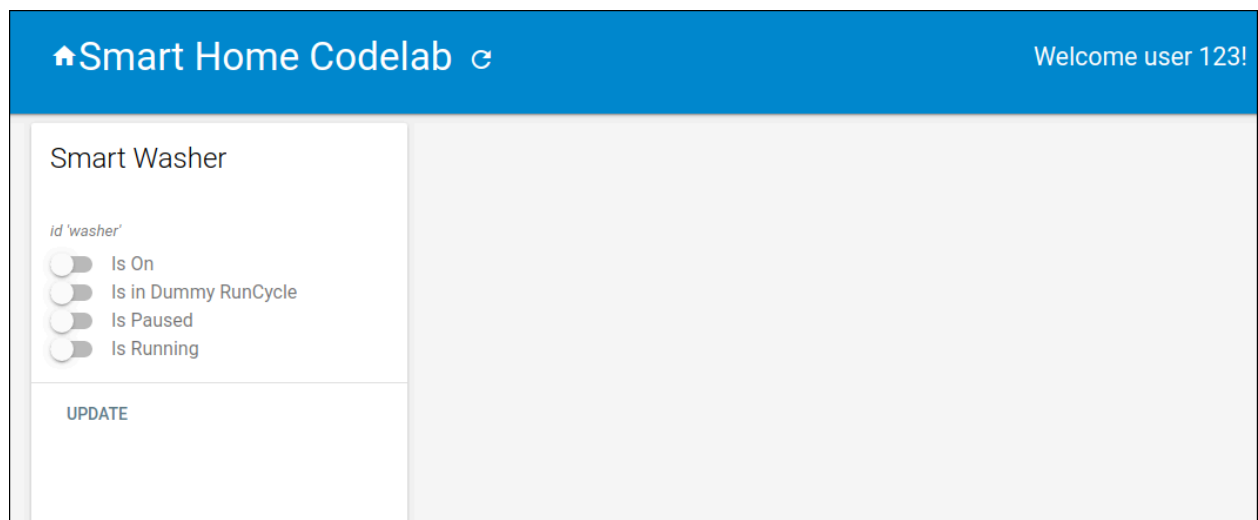
Hosting URL: <https://<project-id>.web.app>

這個指令會部署網頁應用程式，以及多個 [Cloud Functions for Firebase](#)。

**注意：**如果尚未為專案設定帳單，您必須升級至 **Blaze (即付即用)** 方案，[Firebase 函式](#) 才能完成專案部署。

**注意：**專案設定可能需要幾分鐘才能完全傳播。如果嘗試部署時看到「HTTP Error: 403」，請稍候一分鐘再試一次。

在瀏覽器中開啟「Hosting URL」(主機網址) (<https://<project-id>.web.app>)，即可查看網頁應用程式。您會看到下列介面：



這個網頁版 UI 代表第三方平台，可查看或修改裝置狀態。如要開始將裝置資訊填入資料庫，請按一下「更新」。頁面上不會顯示任何變更，但洗衣機的目前狀態會儲存在資料庫中。

注意：虛擬裝置狀態會使用 [Firebase 即時資料庫](#) 儲存。您可以使用 [Firebase 控制台](#) 直接查看資料庫內容。在左側工具列中，依序點選「Build」>「即時資料庫」。洗衣機狀態的任何更新都會顯示在這裡。



現在請使用 [Google Home 開發人員控制台](#)，將部署的雲端服務連結至 Google 助理。

## 設定開發人員控制台專案

在「開發」分頁中，為互動新增「顯示名稱」。這個名稱會顯示在 Google Home 應用程式中。

在「應用程式品牌宣傳」下方，上傳應用程式圖示的 `png` 檔案，大小為 144 x 144 像素，並命名為 `.png`。

如要啟用「帳戶連結」，請使用下列帳戶連結設定：

|        |                                                                                  |
|--------|----------------------------------------------------------------------------------|
| 用戶端 ID | ABC123                                                                           |
| 用戶端密鑰  | DEF456                                                                           |
| 授權網址   | <code>https://us-central1-&lt;project-id&gt;.cloudfunctions.net/fakeauth</code>  |
| 權杖網址   | <code>https://us-central1-&lt;project-id&gt;.cloudfunctions.net/faketoken</code> |

請務必將網址更新為您的專案 ID。

## Account Linking

Authentication allows you to link your users' Google Accounts with their accounts in your authentication system. Then you can identify your users when your fulfillment receives a smart home intent.

 [Start here.](#) Read the OAuth 2.0 account linking guide before continuing.

### OAuth Client ID \*

String that identifies the request origin as Google.

Client ID\*  
ABC123

### Client secret \*

Secret string that you registered with Google for your Client ID.

Client secret\*  
.....

### Authorization URL \*

Endpoint for your sign-in web page that supports OAuth 2.0 code or implicit flows.

Authorization URL\*  
`https://us-central1-<project-id>.cloudfunctions.net/fakeauth`

Please enter a valid https link

### Token URL \*

OAuth 2.0 endpoint for token exchange.

Token URL\*  
`https://us-central1-<project-id>.cloudfunctions.net/faketoken`

Please enter a valid https link

在「Cloud 實現網址」下方，輸入提供智慧住宅意圖實現功能的 Cloud 函式網址。

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

## Cloud fulfillment URL

Enter the fulfillment URL used to process your smart home intents.

Fulfillment URL \*

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

Please enter a valid https link

按一下「儲存」儲存專案設定，然後按一下「下一步：測試」，在專案中啟用測試。

 Developer Console

smarthome-washer-dc

Project home

Matter

Cloud-to-cloud

Resources

Develop

Test

Certify

Launch

Analytics

## Develop

Edit the integration.

| Name           | Status | Last modified ↑       |
|----------------|--------|-----------------------|
| Codelab Washer | Draft  | 5:43 PM - Oct 2, 2024 |

Next: Test

### Related content

#### Get started

Learn how to properly set up your environment to ensure your implementation will be a success.

[Learn more](#)

#### Supported devices

See which devices and traits are supported across apps and services.

[Learn more](#)

#### Codelabs

Get hands-on, step-by-step guidance with our technical product tutorials.

[Go to codelab](#)

現在可以開始實作必要的 Webhook，將裝置狀態與 Google 助理連結。

## 4. 建立洗衣機

整合作業設定完成後，即可新增裝置並傳送資料。雲端服務必須處理下列意圖：

- 當 Google 助理想瞭解使用者已連線的裝置時，就會發生 `SYNC` 意圖。使用者連結帳戶時，系統會將這個權杖傳送至您的服務。您應以 JSON 酬載回應，列出所有使用者的裝置及其功能。
- 當 Google 助理想瞭解裝置的目前狀態時，就會發生 `QUERY` 意圖。您應使用 JSON 酬載回覆，其中包含每個要求裝置的狀態。
- 當 Google 助理想代表使用者控制裝置時，就會發生 `EXECUTE` 意圖。您應以 JSON 酬載回應，其中包含每個要求裝置的執行狀態。
- 使用者取消連結帳戶與 Google 助理時，系統會觸發 `DISCONNECT` 意圖。您應停止將該使用者裝置的事件傳送至 Google 助理。

您會在後續章節中更新先前部署的函式，以處理這些意圖。

### 更新 SYNC 回應

開啟 `functions/index.js`，其中包含用來回應 Google 助理要求的程式碼。

您需要處理 `SYNC` 意圖，方法是傳回裝置中繼資料和功能。更新 `onSync` 陣列中的 JSON，加入洗衣機的裝置資訊和[建議特徵](#)。

`index.js`

```

app.onSync((body) => {
  return {
    requestId: body.requestId,
    payload: {
      agentUserId: USER_ID,
      devices: [{
        id: 'washer',
        type: 'action.devices.types.WASHER',
        traits: [
          'action.devices.traits.OnOff',
          'action.devices.traits.StartStop',
          'action.devices.traits.RunCycle',
        ],
        name: {
          defaultNames: ['My Washer'],
          name: 'Washer',
          nicknames: ['Washer'],
        },
        deviceInfo: {
          manufacturer: 'Acme Co',
          model: 'acme-washer',
          hwVersion: '1.0',
          swVersion: '1.0.1',
        },
        willReportState: true,
        attributes: {
          pausable: true,
        },
      }],
    },
  };
});

```

## 部署至 Firebase

使用 Firebase CLI 部署更新後的雲端執行要求：

```
firebase deploy --only functions
```

## 連結至 Google 助理

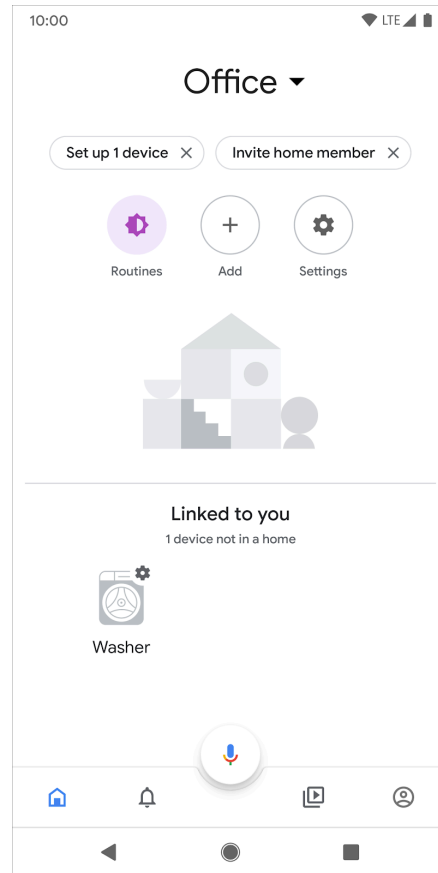
如要測試雲端對雲端整合，您必須將專案連結至 Google 帳戶。這樣一來，您就能透過登入相同帳戶的 Google 助理介面和 Google Home 應用程式進行測試。

**重要事項：**本程式碼研究室包含帳戶連結的實作項目，但不會實際檢查使用者憑證。在實際工作環境中，您應實作 OAuth 2.0 通訊協定，確保裝置安全無虞。

1. 在手機上開啟 Google 助理設定。請注意，您登入的帳戶應與控制台的帳戶相同。

2. 依序前往「Google 助理」>「設定」>「居家控制」(位於「Google 助理」下方)。
3. 按一下右上方的搜尋圖示。
4. 使用 **[test]** 前置字元搜尋測試應用程式，找出特定測試應用程式。
5. 選取該項目。Google 助理接著會向服務進行驗證，並傳送 SYNC 要求，請服務提供使用者的裝置清單。

開啟 Google Home 應用程式，確認是否能看到洗衣機裝置。



**注意：**如果在連結步驟中遇到問題，可以使用 [Cloud Logging](#) 查看錯誤記錄。

步驟 5 · 約 5 分鐘

## 5. 處理指令和查詢

雲端服務向 Google 正確回報洗衣機裝置後，您需要新增要求裝置狀態和傳送指令的功能。

### 處理 QUERY 意圖

**QUERY 意圖** 包含一組裝置。請回報每部裝置的目前狀態。

在 `functions/index.js` 中，編輯 **QUERY** 處理常式，處理意圖要求中包含的目標裝置清單。

#### index.js

```
app.onQuery(async (body) => {
  const {requestId} = body;
  const payload = {
    devices: {},
  };
  const queryPromises = [];
  const intent = body.inputs[0];
  for (const device of intent.payload.devices) {
    const deviceId = device.id;
    queryPromises.push(queryDevice(deviceId)
      .then((data) => {
        // Add response to device payload
        payload.devices[deviceId] = data;
      })
    );
  }
  // Wait for all promises to resolve
  await Promise.all(queryPromises);
  return {
    requestId: requestId,
    payload: payload,
  };
});
```

針對要求中包含的每個裝置，傳回儲存在即時資料庫中的目前狀態。更新 `queryFirebase` 和 `queryDevice` 函式，以傳回洗衣機的狀態資料。

#### index.js

```
const queryFirebase = async (deviceId) => {
  const snapshot = await getFirebaseRef().child(deviceId).once('value');
  const snapshotVal = snapshot.val();
  return {
    on: snapshotVal.OnOff.on,
    isPaused: snapshotVal.StartStop.isPaused,
    isRunning: snapshotVal.StartStop.isRunning,
  };
};

const queryDevice = async (deviceId) => {
  const data = await queryFirebase(deviceId);
  return {
    on: data.on,
    isPaused: data.isPaused,
    isRunning: data.isRunning,
    currentRunCycle: [{
      currentCycle: 'rinse',
      nextCycle: 'spin',
      lang: 'en',
    }],
    currentTotalRemainingTime: 1212,
    currentCycleRemainingTime: 301,
  };
};
```

## 處理 EXECUTE 意圖

**EXECUTE 意圖**會處理更新裝置狀態的指令。回應會傳回每個指令的狀態 (例如 **SUCCESS**、**ERROR** 或 **PENDING**) 和新的裝置狀態。

在 `functions/index.js` 中，編輯 **EXECUTE** 處理常式，處理需要更新的特徵清單，以及每個指令的目標裝置組合：

[index.js](#)

```

app.onExecute(async (body) => {
  const {requestId} = body;
  // Execution results are grouped by status
  const result = {
    ids: [],
    status: 'SUCCESS',
    states: {
      online: true,
    },
  };

  const executePromises = [];
  const intent = body.inputs[0];
  for (const command of intent.payload.commands) {
    for (const device of command.devices) {
      for (const execution of command.execution) {
        executePromises.push(
          updateDevice(execution, device.id)
            .then((data) => {
              result.ids.push(device.id);
              Object.assign(result.states, data);
            })
            .catch(() => functions.logger.error('EXECUTE', device.id)));
      }
    }
  }

  await Promise.all(executePromises);
  return {
    requestId: requestId,
    payload: {
      commands: [result],
    },
  };
});

```

針對每個指令和目標裝置，更新即時資料庫中對應所要求特徵的值。修改 `updateDevice` 函式，更新適當的 Firebase 參照，並傳回更新後的裝置狀態。

[index.js](#)

```

const updateDevice = async (execution, deviceId) => {
  const {params, command} = execution;
  let state; let ref;
  switch (command) {
    case 'action.devices.commands.OnOff':
      state = {on: params.on};
      ref = getFirestoreRef().child(deviceId).child('OnOff');
      break;
    case 'action.devices.commands.StartStop':
      state = params.start
        ? {isRunning: true, isPaused: false}
        : {isRunning: false, isPaused: false};
      ref = getFirestoreRef().child(deviceId).child('StartStop');
      break;
    case 'action.devices.commands.PauseUnpause':
      const data = await queryDevice(deviceId);
      state = (data.isPaused === false && data.isRunning === false)
        ? {isRunning: false, isPaused: false}
        : {isRunning: !params.pause, isPaused: params.pause};
      ref = getFirestoreRef().child(deviceId).child('StartStop');
      break;
  }

  return ref.update(state)
    .then(() => state);
};

```

**附註：** [RunCycle](#) 特徵的 `EXECUTE` 處理常式中沒有任何程式碼，因為這是唯讀特徵，不支援指令。

## 6. 測試整合項目

實作所有三個意圖後，即可測試整合服務是否能控制洗衣機。

### 部署至 Firebase

使用 Firebase CLI 部署更新後的雲端執行要求：

```
firebase deploy --only functions
```

### 測試洗衣機

現在透過手機嘗試下列任一語音指令時，即可查看值變化：

「Ok Google，開啟洗衣機。」

「Ok Google，暫停洗衣機。」

「Ok Google，停止洗衣機。」

你也可以提出問題，查看洗衣機的目前狀態。

「Ok Google，洗衣機開著嗎？」

「Ok Google，洗衣機正在運作嗎？」

「Ok Google，洗衣機目前處於哪個行程？」

您可以在 [Firebase 控制台](#) 的「Functions」部分，查看函式下方顯示的記錄，瞭解這些查詢和指令。如要進一步瞭解 Firebase 記錄，請參閱「[寫入及查看記錄](#)」。

您也可以在 [Google Cloud 控制台](#) 中找到這些查詢和指令，方法是依序前往「Logging」>「Logs Explorer」。如要進一步瞭解 Google Cloud 記錄，請參閱「[透過 Cloud Logging 存取事件記錄檔](#)」。

---

## 7. 向 Google 檢舉更新內容

您已將雲端服務與智慧型住宅意圖完全整合，讓使用者可以控制裝置並查詢裝置的目前狀態。不過，這項實作方式仍缺少讓服務主動將事件資訊 (例如裝置狀態或存在與否的變化) 傳送給 Google 助理的方法。

使用者新增或移除裝置，或是裝置功能變更時，您可以使用「要求同步」觸發新的同步要求。透過「回報狀態」，當使用者實際變更裝置狀態 (例如開啟電燈開關) 或使用其他服務變更狀態時，雲端服務可以主動將裝置狀態傳送至 Home Graph。

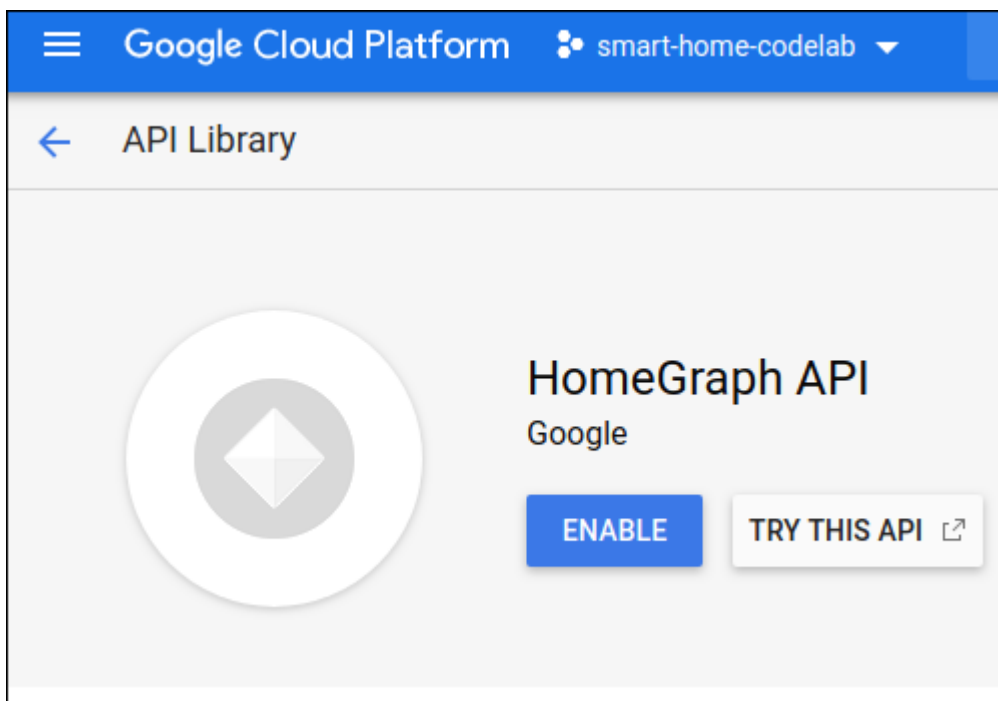
在本節中，您將新增程式碼，從前端網頁應用程式呼叫這些方法。

**注意：**收到特定使用者的 `DISCONNECT` 意圖後，請確保雲端對雲端整合功能停止向 Google 傳送更新。這項意圖表示使用者已取消帳戶與 Google 助理的連結。

### 啟用 HomeGraph API

[HomeGraph API](#) 可讓您在使用者 Home Graph 中儲存及查詢裝置和裝置狀態。如要使用這項 API，請先開啟 Google Cloud 控制台並[啟用 HomeGraph API](#)。

在 Google Cloud 控制台中，請務必選取與整合項目相符的專案 `<project-id>`。接著，在 HomeGraph API 的「API 程式庫」畫面中，按一下「啟用」。



### 啟用回報狀態

寫入即時資料庫會觸發 starter 專案中的 `reportstate` 函式。更新 `functions/index.js` 中的 `reportstate` 函式，擷取寫入資料庫的資料，並使用 Report State 將資料發布至 Home Graph。

[index.js](#)

```

exports.reportstate = functions.database.ref('{deviceId}').onWrite(
  async (change, context) => {
    functions.logger.info('Firebase write event triggered Report State');
    const snapshot = change.after.val();

    const requestBody = {
      requestId: 'ff36a3cc', /* Any unique ID */
      agentUserId: USER_ID,
      payload: {
        devices: {
          states: {
            /* Report the current state of our washer */
            [context.params.deviceId]: {
              on: snapshot.OnOff.on,
              isPaused: snapshot.StartStop.isPaused,
              isRunning: snapshot.StartStop.isRunning,
            },
          },
        },
      },
    };

    const res = await homegraph.devices.reportStateAndNotification({
      requestBody,
    });
    functions.logger.info('Report state response:', res.status, res.data);
  });

```

## 啟用要求同步

在前端網頁 UI 中重新整理圖示，會觸發啟動專案中的 `requestsync` 函式。在 `functions/index.js` 中實作 `requestsync` 函式，呼叫 HomeGraph API。

[index.js](#)

```
exports.requestsync = functions.https.onRequest(async (request, response) => {
  response.set('Access-Control-Allow-Origin', '*');
  functions.logger.info(`Request SYNC for user ${USER_ID}`);
  try {
    const res = await homegraph.devices.requestSync({
      requestBody: {
        agentUserId: USER_ID,
      },
    });
    functions.logger.info('Request sync response:', res.status, res.data);
    response.json(res.data);
  } catch (err) {
    functions.logger.error(err);
    response.status(500).send(`Error requesting sync: ${err}`);
  }
});
```

## 部署至 Firebase

使用 Firebase CLI 部署更新後的程式碼：

```
firebase deploy --only functions
```

## 測試實作成果

在網頁 UI 中按一下「重新整理」



按鈕，並確認 Firebase 控制台記錄中顯示同步要求。

接著，在前端網頁 UI 中調整洗衣機裝置的屬性，然後按一下「更新」。確認您可以在 Firebase 控制台記錄中，查看向 Google 回報的狀態變更。

---

## 8. 恭喜



恭喜！您已使用雲端對雲端整合功能，成功將 Google 助理與裝置雲端服務整合。

## 瞭解詳情

以下提供幾個可深入瞭解的構想：

- 在裝置中新增 [模式](#) 和 [切換按鈕](#)。
- 在裝置中新增更多 [支援的特徵](#)。
- 探索智慧型住宅的 [本機執行](#) 功能。
- 如要瞭解詳情，請參閱 [GitHub 範例](#)。

您也可以進一步瞭解如何 [測試及提交](#) 整合功能以供審查，包括將整合功能發布給使用者的認證程序。

---